



The Poseidon16 Instruction Set Manual, Volume I

Unprivileged Architecture

Table of Contents

1. Architecture Summary	3
2. Register File, Instruction Length, Memory & Alignment.....	3
2.1 Register File.....	3
2.2 Instruction Length	3
2.3 Alignment Rules	3
3. Poseidon16 16-bit Encoding Formats	4
3.1 R-Type - Register/Register and M-extension operations.....	4
3.2 I-Type - Register/Immediate, Load and Store	4
3.3 C-Type - Control Flow	4
4. Field Legend	4
5. Instruction Encoding Table	5
6. Instructions definitions	6
6.1 Add.....	6
6.2 Sub.....	7
6.3 And.....	8
6.4 OR	9
6.5 SLT	10
6.6 MUL.....	11
6.7 DIV	12
6.8 REM	13
6.9 ADDI	14
6.10 ANDI	16
6.11 ORI	17
6.12 SLTI	18
6.13 LW	19
6.14 SW.....	20
6.15 BEQ	21
6.16 JAL	22

1. Architecture Summary

Register File	16 general-purpose registers R0-R15, each register is 32 bits. R0 is hardwired to 0: reads return 0 and writes have no effect.
Instruction Length	Fixed 16-bit (2-byte) instructions.
Memory & Alignment	Byte-addressable memory, Big-Endian byte ordering. Misaligned multi-byte loads/stores are allowed.
Encoding Formats	3 primary formats: R-Type (register-register), I-Type (register-immediate / load-store), and C-Type (control flow).
Mandatory Instructions	Base integer subset: ADD, ADDI, SUB, AND, ANDI, OR, ORI, SLT, SLTI, LW, SW, BEQ, JAL. M extension: MUL, DIV, REM.
Normal PC Advance	PC + 2 bytes for a non-taken branch or normal sequential execution (derived directly from the fixed 16-bit instruction length).

2. Register File, Instruction Length, Memory & Alignment

2.1. Register File

Name	Register No.	Width	Use
R0	0	32-bit	Constant zero register. Read value is always 0. Writes are discarded.
R1-R15	1-15	32-bit	General-purpose registers.

2.2. Instruction Length

Every Poseidon16 instruction occupies exactly 16 bits = 2 bytes. Fetch therefore reads two consecutive bytes and combines them in Big-Endian order: instruction[15:8] comes from memory[PC] and instruction[7:0] comes from memory[PC+1].

Example	Address A	A+1	A+2	A+3	Meaning
32-bit word 0xA1B2C3D4	A1	B2	C3	D4	Big-Endian: most-significant byte at lowest address
Misaligned LW at A+1	B2	C3	D4	next byte	Allowed

2.3. Alignment Rules

Access	Alignment Requirement	Poseidon16 Behavior
Instruction fetch	2-byte instruction boundary	Assuming execution begins at an even PC and all branch/jump displacements are multiples of 2, the PC remains even.
LW / SW word access	No data alignment restriction	Any byte address is legal. Four bytes are read/written in Big-Endian order.
Branch / jump target	2-byte instruction boundary	Stores PC-relative control offsets in 2-byte units, so targets remain instruction-aligned.

3. Poseidon16 16-bit Encoding Formats

Each register specifier requires 4 bits. Poseidon16 uses a 4-bit primary opcode, leaving 12 bits whose interpretation depends on the instruction format.

3.1.R-Type - Register/Register and M-extension operations

Used by ADD, SUB, AND, OR, SLT, MUL, DIV, and REM.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op				rd				rs1				rs2			

3.2.I-Type - Register/Immediate, Load and Store

For arithmetic-immediate and LW, bits 11:8 is rd.

For SW, bits 11:8 are rs2. imm4 is a signed 4-bit immediate (range -8 to +7).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op				rd / rs2				rs1				imm4			

3.3.C-Type - Control Flow

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op				A				B				C			

BEQ interprets A=rs1, B=rs2, C=off4.

JAL interprets A=rd and concatenates B:C as off8[7:0].

Offset denotes the signed byte displacement from the current PC. Because Poseidon16 instructions are 2-byte aligned, the encoded off4/off8 fields store offset / 2. Therefore offset = SignExt(offN) << 1.

Instruction	op	bits 11:8	bits 7:4	bits 3:0	PC-relative interpretation
BEQ	1110	rs1	rs2	off4	byte_offset = SignExt(off4) << 1; range -16..+14 bytes
JAL	1111	rd	off8[7:4]	off8[3:0]	byte_offset = SignExt(off8) << 1; range -256..+254 bytes

4. Field Legend

Field	Meaning
op	4-bit primary opcode in instruction[15:12]. Every mandatory Poseidon16 instruction has a unique opcode.
rd	4-bit destination register number R0-R15.
rs1 / rs2	4-bit source register numbers R0-R15.
imm4	Signed 4-bit two's-complement immediate. Sign-extended to 32 bits before ALU/address use.
off4 / off8	Signed control-flow displacement stored in 2-byte instruction units. Shift left by 1 before adding to PC.
EA	Effective address = R[rs1] + SignExt(imm4). Misalignment is allowed for Poseidon16.
Big-Endian word	For LW/SW, the byte at EA is bits 31:24, EA+1 is 23:16, EA+2 is 15:8, and EA+3 is 7:0.

5. Instruction Encoding Table

op	Type	Instruction		Description	Operation
0000 (0x0)	R	add	rd, rs1, rs2	add	$R[rd] = R[rs1] + R[rs2]$
0001 (0x1)	R	sub	rd, rs1, rs2	subtract	$R[rd] = R[rs1] - R[rs2]$
0010 (0x2)	R	and	rd, rs1, rs2	bitwise and	$R[rd] = R[rs1] \& R[rs2]$
0011 (0x3)	R	or	rd, rs1, rs2	bitwise or	$R[rd] = R[rs1] R[rs2]$
0100 (0x4)	R	slt	rd, rs1, rs2	set less than (signed)	$R[rd] = (\text{Signed}(R[rs1]) < \text{Signed}(R[rs2])) ? 1 : 0$
0101 (0x5)	R	mul	rd, rs1, rs2	signed multiply, low 32 bits	$R[rd] = (R[rs1] * R[rs2])[31:0]$
0110 (0x6)	R	div	rd, rs1, rs2	signed divide	$R[rd] = (R[rs2] == 0) ? 0xFFFFFFFF : \text{Signed}(R[rs1]) / \text{Signed}(R[rs2])$
0111 (0x7)	R	rem	rd, rs1, rs2	signed remainder	$R[rd] = (R[rs2] == 0) ? R[rs1] : \text{Signed}(R[rs1]) \% \text{Signed}(R[rs2])$
1000 (0x8)	I	addi	rd, rs1, imm4	add immediate	$R[rd] = R[rs1] + \text{SignExt}(\text{imm4})$
1001 (0x9)	I	andi	rd, rs1, imm4	and immediate	$R[rd] = R[rs1] \& \text{SignExt}(\text{imm4})$
1010 (0xA)	I	ori	rd, rs1, imm4	or immediate	$R[rd] = R[rs1] \text{SignExt}(\text{imm4})$
1011 (0xB)	I	slti	rd, rs1, imm4	set less than immediate (signed)	$R[rd] = (\text{Signed}(R[rs1]) < \text{SignExt}(\text{imm4})) ? 1 : 0$
1100 (0xC)	I	lw	rd, imm4(rs1)	load 32-bit word	$EA = R[rs1] + \text{SignExt}(\text{imm4}); R[rd] = \text{Mem32_BE}[EA]$
1101 (0xD)	I	sw	rs2, imm4(rs1)	store 32-bit word	$EA = R[rs1] + \text{SignExt}(\text{imm4}); \text{Mem32_BE}[EA] = R[rs2]$
1110 (0xE)	C	beq	rs1, rs2, offset	branch if equal	if $R[rs1] == R[rs2]$: $PC = PC + (\text{SignExt}(\text{off4}) \ll 1)$; else $PC = PC + 2$
1111 (0xF)	C	jal	rd, offset	jump and link	$R[rd] = PC + 2; PC = PC + (\text{SignExt}(\text{off8}) \ll 1)$

6. Instructions definitions

6.1.ADD

6.1.1Assembler Format

ADD rd, rs1, rs2

6.1.2Encoding



6.1.3Operation

$R[rd] = R[rs1] + R[rs2]$

6.1.4Description

Adds the 32-bit value in source register rs1 to the 32-bit value in source register rs2 and writes the result to rd. Use ADD when both operands are already in registers. If rd is R0, the calculated result is discarded because R0 is hardwired to zero.

RULE	Operands: any registers R0–R15.
RULE	R0 can be used as a zero-source operand.

6.1.5Examples

ADD R2, R3, R4	If R3 = 10 and R4 = 7, then R2 ← 17.
Machine code	0000 0010 0011 0100 (0x0234)

ADD R5, R5, R0	R5 is unchanged because R0 reads as 0.
Machine code	0000 0101 0101 0000 (0x0550)

6.2.SUB

6.2.1Assembler Format

SUB rd, rs1, rs2

6.2.2Encoding



6.2.3Operation

$R[rd] = R[rs1] - R[rs2]$

6.2.4Description

Subtracts the value in rs2 from the value in rs1 and stores the result in rd. Operand order matters: rs1 is the value being reduced, and rs2 is the value subtracted from it.

RULE	Use SUB for register-to-register subtraction.
RULE	Writing rd = R0 discards the result.

Examples

SUB R5, R6, R7	If R6 = 20 and R7 = 8, then $R5 \leftarrow 12$.
Machine code	0001 0101 0110 0111 (0x1567)

SUB R3, R0, R4	Computes $0 - R4$, useful for forming a negated value.
Machine code	0001 0011 0000 0100 (0x1304)

6.3.AND

6.3.1Assembler Format

AND rd, rs1, rs2

6.3.2Encoding



6.3.3Operation

$R[rd] = R[rs1] \& R[rs2]$

6.3.4Description

Performs a bit-by-bit AND of rs1 and rs2. Each result bit is 1 only when the corresponding bit is 1 in both source registers. Use AND for masking, clearing selected bits, or testing bit patterns.

RULE	Operation is bitwise over the 32-bit register values.
RULE	R0 may be used to force a zero result.

6.3.5Examples

AND R2, R3, R4	If R3 = 0x0000F0F0 and R4 = 0x0000FF0, then R2 $\leftarrow$ 0x00000F0.
Machine code	0010 0010 0011 0100 (0x2234)

AND R1, R2, R0	R1 $\leftarrow$ 0.
Machine code	0010 0001 0010 0000 (0x2120)

6.4.OR

6.4.1Assembler Format

OR rd, rs1, rs2

6.4.2Encoding



6.4.3Operation

$R[rd] = R[rs1] \mid R[rs2]$

6.4.4Description

Performs a bit-by-bit OR of rs1 and rs2. Each result bit is 1 when either corresponding source bit is 1. Use OR to set or combine bit fields.

RULE	Operation is bitwise over the 32-bit register values.
RULE	OR with R0 copies the other source value into rd.

6.4.5Examples

OR R2, R3, R4	If R3 = 0x0000F000 and R4 = 0x000000F0, then R2 ← 0x0000F0F0.
Machine code	0011 0010 0011 0100 (0x3234)

OR R1, R5, R0	R1 receives the value currently in R5.
Machine code	0011 0001 0101 0000 (0x3150)

6.5.SLT

6.5.1Assembler Format

SLT rd, rs1, rs2

6.5.2Encoding



6.5.3Operation

$R[\text{rd}] = (\text{Signed}(R[\text{rs1}]) < \text{Signed}(R[\text{rs2}])) ? 1 : 0$

6.5.4Description

Compares rs1 and rs2 as signed 32-bit values. If rs1 is strictly less than rs2, rd receives 1; otherwise rd receives 0. Use SLT to convert a signed comparison into a 0/1 register value.

RULE	The comparison is signed.
RULE	The result is exactly 0 or 1.

6.5.5Examples

SLT R1, R2, R3	If $R2 = -5$ and $R3 = 2$, then $R1 \leftarrow 1$.
Machine code	0100 0001 0010 0011 (0x4123)

SLT R4, R5, R5	A value is never less than itself, so $R4 \leftarrow 0$.
Machine code	0100 0100 0101 0101 (0x4455)

6.6.MUL

6.6.1Assembler Format

MUL rd, rs1, rs2

6.6.2Encoding



6.6.3Operation

$R[rd] = (R[rs1] * R[rs2])[31:0]$

6.6.4Description

Multiplies the two source values using signed multiplication and stores only the lower 32 bits of the product in rd. Use MUL for integer products when the low 32-bit result is the required result.

RULE	MUL is part of the required M extension.
RULE	Only product bits [31:0] are written to rd.

6.6.5Examples

MUL R4, R5, R6	If R5 = 6 and R6 = 7, then R4 ← 42.
Machine code	0101 0100 0101 0110 (0x5456)

MUL R2, R3, R0	Any value multiplied by R0 gives R2 ← 0.
Machine code	0101 0010 0011 0000 (0x5230)

6.7.DIV

6.7.1Assembler Format

DIV rd, rs1, rs2

6.7.2Encoding

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op 0110				rd quotient				rs1 dividend				rs2 divisor			

6.7.3Operation

if (R[rs2] == 0)
 R[rd] = 0xFFFFFFFF
else
 R[rd] = Signed(R[rs1]) / Signed(R[rs2])

6.7.4Description

Divides signed rs1 by signed rs2 and writes the quotient to rd. The Poseidon16 definition explicitly specifies division-by-zero behavior: rd receives 0xFFFFFFFF (all 1s).

RULE	M-extension signed divide.
RULE	Division by zero does not trap in the provided Poseidon16 definition; it returns 0xFFFFFFFF.

6.7.5Examples

DIV R7, R8, R9	If R8 = 20 and R9 = 4, then R7 ← 5.
Machine code	0110 0111 1000 1001 (0x6789)

6.8.REM

6.8.1Assembler Format

REM rd, rs1, rs2

6.8.2Encoding



6.8.3Operation

```
if (R[rs2] == 0)
    R[rd] = R[rs1]
else
    R[rd] = Signed(R[rs1]) % Signed(R[rs2])
```

6.8.4Description

Computes the signed remainder of rs1 divided by rs2 and stores it in rd. If the divisor is zero, the Poseidon16 definition returns the original dividend value R[rs1].

RULE	M-extension signed remainder.
RULE	Division-by-zero case returns R[rs1] rather than raising an exception.

6.8.5Examples

REM R10, R11, R12	If R11 = 20 and R12 = 3, then R10 ← 2.
Machine code	0111 1010 1011 1100 (0x7ABC)

REM R2, R3, R0	Since the divisor is zero, R2 ← R3.
Machine code	0111 0010 0011 0000 (0x7230)

6.9.ADDI

6.9.1Assembler Format

ADDI rd, rs1, imm4

6.9.2Encoding



6.9.3Operation

$R[rd] = R[rs1] + \text{SignExt}(\text{imm4})$

6.9.4Description

Adds a small signed constant to rs1. The 4-bit immediate is interpreted as two's-complement in the range -8 to +7 and sign-extended to 32 bits before addition. Use ADDI for small increments, decrements, and address adjustments.

RULE	imm4 range: -8 ... +7.
RULE	Examples: +7 = 0111; -3 = 1101.

6.9.5Examples

ADDI R2, R3, -3	If R3 = 10, then R2 ← 7.
Machine code	1000 0010 0011 1101 (0x823D)

ADDI R5, R5, 1	Increments R5 by 1.
Machine code	1000 0101 0101 0001 (0x8551)

6.10.ANDI

6.10.1Assembler Format

ANDI rd, rs1, imm4

6.10.2Encoding



6.10.3Operation

$R[rd] = R[rs1] \& \text{SignExt}(\text{imm4})$

6.10.4Description

Performs bitwise AND between rs1 and a sign-extended 4-bit immediate. Because the immediate is sign-extended, negative immediate values produce 1s in the upper bits. Use ANDI for small immediate masks.

RULE	imm4 is signed and sign-extended, not zero-extended.
RULE	-1 encodes as 1111 and sign-extends to 0xFFFFFFFF.

6.10.5Examples

ANDI R2, R3, -1	The immediate becomes 0xFFFFFFFF, so R2 receives the same bit pattern as R3.
Machine code	1001 0010 0011 1111 (0x923F)

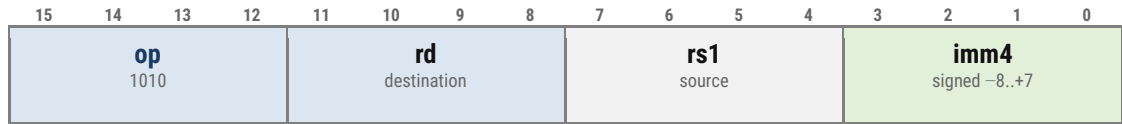
ANDI R4, R5, 6	Masks R5 with sign-extended 0x00000006.
Machine code	1001 0100 0101 0110 (0x9456)

6.11.ORI

6.11.1Assembler Format

ORI rd, rs1, imm4

6.11.2Encoding



6.11.3Operation

$R[rd] = R[rs1] \mid \text{SignExt}(\text{imm4})$

6.11.4Description

Performs bitwise OR between rs1 and the sign-extended 4-bit immediate. Use ORI to set bits using a small immediate mask. Remember that negative immediates sign-extend with 1s in the upper bits.

RULE	imm4 range: -8 ... +7.
RULE	The immediate is sign-extended to 32 bits before OR.

6.11.5Examples

ORI R4, R5, 3	R4 receives R5 OR 0x00000003.
Machine code	1010 0100 0101 0011 (0xA453)

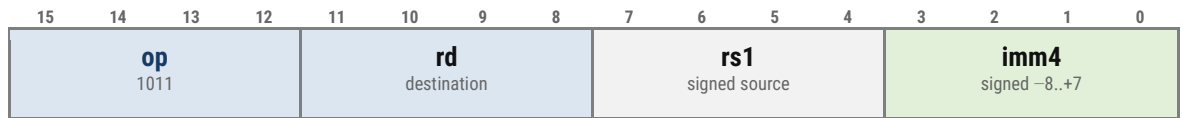
ORI R6, R7, -1	The immediate sign-extends to all 1s, so R6 ← 0xFFFFFFFF.
Machine code	1010 0110 0111 1111 (0xA67F)

6.12.SLTI

6.12.1Assembler Format

SLTI rd, rs1, imm4

6.12.2Encoding



6.12.3Operation

$R[\text{rd}] = (\text{Signed}(R[\text{rs1}]) < \text{SignExt}(\text{imm4})) ? 1 : 0$

6.12.4Description

Compares rs1 as a signed 32-bit value against the sign-extended signed imm4. If rs1 is smaller, rd receives 1; otherwise it receives 0. Use SLTI for comparisons against small constants.

RULE	Signed comparison.
RULE	Immediate range: -8 ... +7.

6.12.5Examples

SLTI R6, R7, 0	If R7 = -3, then R6 ← 1 because -3 < 0.
Machine code	1011 0110 0111 0000 (0xB670)

SLTI R2, R3, -2	If R3 = 5, then R2 ← 0.
Machine code	1011 0010 0011 1110 (0xB23E)

6.13.LW

6.13.1Assembler Format

LW rd, imm4(rs1)

6.13.2Encoding



6.13.3Operation

$EA = R[rs1] + \text{SignExt}(imm4)$
 $R[rd] = \text{Mem32_BE}[EA]$

6.13.4Description

Forms the effective byte address EA by adding the signed imm4 to the base register rs1, then loads four consecutive bytes into rd in Big-Endian order. Poseidon16 explicitly allows misaligned word loads, so EA does not have to be divisible by 4.

RULE	Byte at EA → result bits 31:24. EA+1 → 23:16; EA+2 → 15:8. EA+3 → 7:0.
RULE	imm4 is a byte offset in the range -8 ... +7.
RULE	Any byte address is legal for LW in Poseidon16.

6.13.5Examples

LW R2, 1(R3)	If R3 = 0x1000 and memory[1001..1004] = A1 B2 C3 D4, then $R2 \leftarrow 0xA1B2C3D4$. EA = 0x1001 is misaligned but allowed.
Machine code	1100 0010 0011 0001 (0xC231)

LW R4, -2(R5)	EA = R5 - 2; load four bytes starting at that byte address.
Machine code	1100 0100 0101 1110 (0xC45E)

6.14.SW

6.14.1Assembler Format

SW rs2, imm4(rs1)

6.14.2Encoding



6.14.3Operation

$EA = R[rs1] + \text{SignExt}(imm4)$
 $\text{Mem32_BE}[EA] = R[rs2]$

6.14.4Description

Forms EA from the base register rs1 plus signed imm4, then stores the 32-bit value in rs2 into four consecutive bytes in Big-Endian order. Unlike arithmetic/load I-type instructions, bits 11:8 hold rs2 rather than rd. Misaligned stores are allowed.

RULE	R[rs2] bits 31:24 → memory[EA]; bits 23:16 → EA+1; bits 15:8 → EA+2; bits 7:0 → EA+3.
RULE	imm4 is a signed byte offset, -8 ... +7.
RULE	Any byte address is legal for SW in Poseidon16.

6.14.5Examples

SW R4, -1(R5)	If R5 = 0x2002 and R4 = 0x11223344, then EA = 0x2001 and bytes 11 22 33 44 are stored at 0x2001..0x2004.
Machine code	1101 0100 0101 1111 (0xD45F)

SW R2, 4(R3)	Stores R2 at the byte address R3 + 4.
Machine code	1101 0010 0011 0100 (0xD234)

6.15.BEQ

6.15.1Assembler Format

BEQ rs1, rs2, offset

6.15.2Encoding



6.15.3Operation

```
if (R[rs1] == R[rs2])
    PC = PC + (SignExt(off4) << 1)
else
    PC = PC + 2
```

6.15.4Description

Compares two registers. If they are equal, control branches relative to the current PC. The encoded off4 is not a raw byte count: it is a signed 4-bit displacement in 2-byte instruction units, so the byte displacement is $\text{SignExt}(\text{off4}) \ll 1$. If the registers are not equal, execution continues at the next 16-bit instruction.

RULE	off4 range: -8 ... +7 instruction units.
RULE	Byte branch range: -16 ... +14 bytes.
RULE	Targets remain on 2-byte instruction boundaries.

6.15.5Examples

BEQ R1, R2, +6 bytes	At PC = 0x0100, +6 bytes requires off4 = +3 = 0011. If R1 = R2, new PC = 0x0106; otherwise PC = 0x0102.
Machine code	1110 0001 0010 0011 (0xE123)

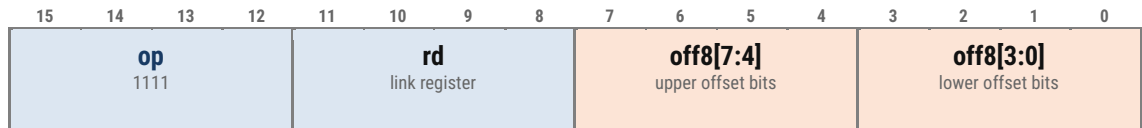
BEQ R3, R4, -4 bytes	-4 bytes requires off4 = -2 = 1110.
Machine code	1110 0011 0100 1110 (0xE34E)

6.16.JAL

6.16.1Assembler Format

JAL rd, offset

6.16.2Encoding



6.16.3Operation

$R[rd] = PC + 2$

$PC = PC + (\text{SignExt}(\text{off8}) \ll 1)$

6.16.4Description

Saves the return address (the address of the next 16-bit instruction) into rd, then jumps to a PC-relative target. The 8-bit signed offset is stored in 2-byte instruction units, so it is sign-extended and shifted left by 1 before being added to PC.

RULE	off8 range: -128 ... +127 instruction units.
RULE	Byte jump range: -256 ... +254 bytes.
RULE	If rd = R0, the link value is discarded, so the instruction acts as an unconditional PC-relative jump.

6.16.5Examples

JAL R1, +20 bytes	At PC = 0x0200, +20 bytes means off8 = +10 = 00001010. R1 ← 0x0202 and PC ← 0x0214.
Machine code	1111 0001 0000 1010 (0xF10A)

JAL R0, -8 bytes	-8 bytes means off8 = -4 = 11111100. The return address write is discarded because rd = R0.
Machine code	1111 0000 1111 1100 (0xF0FC)